

ISM2411 — Lab Week 11

Customer Dictionary & Lookup — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 11 · Unit 3 · Data Structures

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Segments	3
6 Segment-by-Segment Walkthrough	4
6.1 Intro (0:00–0:04)	4
6.2 Exercise 1 — One Customer (0:04–0:11, 7 min)	4
6.3 Exercise 2 — Update and Add (0:11–0:18, 7 min)	6
6.4 Exercise 3 — Safe Lookup (0:18–0:27, 9 min)	8
6.5 Exercise 4 — Iterate (0:27–0:34, 7 min)	10
6.6 Exercise 5 — Lookup Table (0:34–0:42, 8 min)	12
6.7 Exercise 6 — List of Dicts (0:42–0:50, 8 min)	14
6.8 Exercise 7 — Nested Catalog (0:50–0:58, 8 min)	16
6.9 Exercise 8 — Sales by Region (0:58–1:07, 9 min)	18
7 Wrap-Up (last ~8 minutes)	20
8 Appendix A — Full Answer Key (customers.py)	20
9 Appendix B — Extra Practice (only if the class finishes early)	22

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 11 Lab — Customer Dictionary & Lookup
Unit	Unit 3 · Data Structures
Class length	75 minutes
Format	Live code-along
Prerequisites	Module 10: lists, indexing; Modules 05–06: conditionals, loops, accumulator pattern
Student-facing lab page	markumreed.github.io/ism2411 — week11_lab
Exercises covered	Exercises 1–8 (required)
Submission	customers.py via GitHub (week11/ folder), repo URL to Canvas

Where Module 10's lists are reached by *position* (index 0, 1, 2...), this module's dictionaries are reached by *name* — a genuinely different mental model, and arguably the single most professionally useful data structure in the entire course. The lab page's own framing is exactly right: “this is the pattern that underlies every real CRM and analytics workflow” — a customer record with named fields, a lookup table, a list of records, and a summary built by accumulating into a dictionary are all patterns students will meet again, immediately, in any real business-data role. Protect real time for Exercise 3's `.get()` vs. bracket-notation distinction and Exercise 8's accumulate-into-a-dictionary pattern — both come up constantly in real work.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Create a dictionary with named fields, and access, add, update, and delete fields using key syntax.
2. Explain the difference between `.get()` (safe, with a default) and bracket notation (`KeyError` if missing) for dictionary lookups, and know when to use each.
3. Iterate over a dictionary's key/value pairs with `.items()`.
4. Use a dictionary as a **lookup table** — mapping a category (like a tier name) to a value (like a discount rate).
5. Build a summary dictionary from a list of records, accumulating values by key with `.get(key, 0)` inside a loop.

3 Before Class — Setup Checklist

-
- ☐ Open `customers.py`, empty except the header comment.
 - ☐ Decide on one consistent example customer (this guide uses Alice, gold tier) to build through Exercises 1–5, so the room isn't re-orienting to new data every exercise.
 - ☐ Pre-draw a quick side-by-side board sketch contrasting a list's `[0]`, `[1]`, `[2]` access with a dictionary's `["name"]`, `["email"]`, `["tier"]` access — this single visual anchors the whole “position vs. name” framing for the session.

4 Materials Needed

- Instructor laptop + terminal + editor, Python 3.10+
- Students: `customers.py`, GitHub repo with a `week11/` folder to add

5 Segments

The required exercises fill the available time.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Last module, a list held five items, reached by position — index 0, 1, 2. Today, a dictionary holds fields reached by name — `customer["name"]`, `customer["email"]`, `customer["tier"]`. This is a genuinely different way of organizing data, and it’s the single structure you’ll see most often in real business software — every customer record, every product catalog, every API response you’ll ever work with professionally is built on this pattern.”

Do: Open `customers.py`, type the header:

```
# customers.py
# ISM2411 Module 11 Lab – Customer Dictionary & Lookup
```

6.2 Exercise 1 — One Customer (0:04–0:11, 7 min)

Teaching goal: Dictionary literal syntax — curly braces, key: value pairs — and accessing a field by name with bracket notation.

Say to the class:

“Curly braces this time, not square brackets. Each field gets a name — a ‘key’ — and a value, separated by a colon.”

Live-code this:

```
# --- Exercise 1 ---
customer = {
    "name": "Alice",
    "email": "alice@example.com",
    "tier": "gold",
    "ytd_spend": 1200,
}
print(customer["name"])
print(customer["email"])
print(customer["tier"])
print(customer["ytd_spend"])
print(customer)
```

Line-by-line explanation:

- `{"name": "Alice", "email": "alice@example.com", ...}` — curly braces `{...}` create a **dictionary**; each entry is a "key": value pair, separated by commas, exactly parallel to a list's comma-separated items — but here, each value has a *name* attached to it rather than just a position. Note the formatting across multiple lines with a trailing comma after the last entry — mention explicitly that this multi-line style is just for readability; the whole thing could be written on one line and mean exactly the same thing.
- `customer["name"]` — bracket notation, but with a **string key** instead of a numeric index — say explicitly, contrasting directly with Module 10: `inventory[0]` asked “what’s at position 0?”; `customer["name"]` asks “what’s stored under the key 'name'?” — a fundamentally different kind of question, even though the bracket syntax looks similar.
- `print(customer)` — printing the whole dictionary shows all key/value pairs together, in curly-brace notation, in the order they were originally added — worth a brief note that **dictionaries in modern Python (3.7+) preserve insertion order**, which wasn’t always guaranteed in much older Python versions; not something students need to worry about practically, but a good “why does the order look predictable” answer if asked.

Run it. Expected output:

```
Alice
alice@example.com
gold
1200
{'name': 'Alice', 'email': 'alice@example.com', 'tier': 'gold', 'ytd_spend': 1200}
```

Common student mistakes to watch for:

- Using `=` instead of `:` inside the dictionary literal (a natural habit from variable assignment) — produces a `SyntaxError`; worth a quick side-by-side comparison of `customer["name"] = "Alice"` (valid — this is how you’d *update* a field, coming in Exercise 2) versus `{"name" = "Alice"}` (invalid inside a literal) to clarify when each symbol is appropriate.
- Forgetting quotes around a key (`customer[name]` instead of `customer["name"]`) — Python interprets `name` as a variable name, not a string, and raises a `NameError` if no variable called `name` happens to exist (or, worse, silently uses the wrong value if one does) — worth flagging as another instance of the “unquoted text is a variable reference, not a string” rule from Module 01.

Check for understanding: “What would `customer["Name"]` (capital N) return?” (A `KeyError` — dictionary keys are case-sensitive strings, exactly like any other string comparison in Python; `"Name"` and `"name"` are different keys entirely, even though they look almost identical to a human reader.)

6.3 Exercise 2 — Update and Add (0:11–0:18, 7 min)

Teaching goal: Add a new field, update an existing one, and delete one with `del` — three distinct operations, all using the same bracket-notation syntax students already know from Exercise 1.

Say to the class:

“Three changes to the same dictionary — adding a field that wasn’t there before, changing one that was, and deleting one entirely. Print after each so you can watch it evolve.”

Live-code this:

```
# --- Exercise 2 ---
customer["phone"] = "555-1234"
print(customer)

customer["tier"] = "platinum"
print(customer)

del customer["email"]
print(customer)
```

Line-by-line explanation:

- `customer["phone"] = "555-1234"` — assigning to a key that **doesn’t yet exist** adds it as a new field — there’s no separate “add” syntax; adding and updating use identical syntax, and Python decides which one happens based on whether the key was already present.
- `customer["tier"] = "platinum"` — assigning to a key that **does** already exist *overwrites* its value — same syntax as the line above, different outcome, purely because “tier” was already a key and “phone” wasn’t. This symmetry is worth naming explicitly: **the same one line of code means “add” or “update” depending only on whether the key already exists** — there’s nothing in the syntax itself that distinguishes the two operations.
- `del customer["email"]` — `del` removes a key (and its value) entirely from the dictionary. Contrast this explicitly with just setting a field to an empty string or `None` — `del` genuinely removes the key, so a subsequent `customer["email"]` lookup will raise `KeyError`, exactly like the key was never there, whereas `customer["email"] = None` would leave the key present with an empty-ish value, a meaningfully different state (Exercise 3 explores exactly this distinction).

Run it. Expected output (three separate prints):

```
{'name': 'Alice', 'email': 'alice@example.com', 'tier': 'gold', 'ytd_spend': 1200, 'phone': '555-1234'}
{'name': 'Alice', 'email': 'alice@example.com', 'tier': 'platinum', 'ytd_spend': 1200, 'phone': '555-1234'}
{'name': 'Alice', 'tier': 'platinum', 'ytd_spend': 1200, 'phone': '555-1234'}
```

Common student mistakes to watch for:

- Trying `customer.delete("email")` (a method name borrowed from another language or a guess) instead of the `del` keyword — `del` is a Python keyword, not a dictionary method, and it operates on the *access expression* (`customer["email"]`), not by being called on the dictionary itself. This distinction (keyword-with-an-expression vs. method-call syntax) is worth stating explicitly since it's genuinely inconsistent-looking the first time.
- Attempting to `del` a key that doesn't exist — raises `KeyError`, same error family as looking up a missing key; a good moment to note that `del` and bracket-notation lookup share the same failure mode for the same underlying reason (both need the key to already be present).

Check for understanding: “After these three operations, does the dictionary have more, fewer, or the same number of keys compared to Exercise 1’s original four?” (The same — four: `name`, `tier`, `ytd_spend` remain from the original set, `email` was removed, `phone` was added — one removed, one added, net unchanged. Good arithmetic check that the room is tracking the dictionary’s actual state, not just the individual operations in isolation.)

6.4 Exercise 3 — Safe Lookup (0:18–0:27, 9 min)

Teaching goal: The most important distinction in this lab: `.get()` (returns a default instead of crashing) versus bracket notation (`KeyError` if the key is missing) — and when a professional would choose each.

Say to the class:

“You just deleted email from this dictionary. What happens if code elsewhere in a larger program tries to look it up, not knowing it was deleted? Two different ways to ask, with two very different outcomes.”

Live-code this:

```
# --- Exercise 3 ---
print(customer.get("email", "no email on file"))

try:
    print(customer["email"])
except KeyError as e:
    print(f"KeyError caught: {e}")
```

Line-by-line explanation:

- `customer.get("email", "no email on file")` — `.get()` is a dictionary **method** that takes the key you want, plus a **default value** to return if that key isn't present. Since "email" was deleted in Exercise 2, this returns the default string instead of raising any error at all — the program keeps running smoothly, with an explicit, readable fallback value.
- `customer["email"]` — bracket notation, same as every prior exercise — but this time, since the key genuinely doesn't exist, it raises `KeyError: 'email'`, wrapped here in a `try/except` (Module 10 Exercise 5's pattern) so the program doesn't crash outright.
- **The decision rule to state explicitly, since it's the exercise's real point:** use `.get()` with a sensible default when a missing key is a normal, expected possibility your code should handle gracefully (a customer who genuinely has no phone number on file, say); use bracket notation — and let a missing key raise a loud `KeyError` — when a missing key would indicate a genuine bug or data-integrity problem you *want* to know about immediately, rather than silently paper over with a default.

Run it. Expected output:

```
no email on file
KeyError caught: 'email'
```

Common student mistakes to watch for:

- Forgetting the second argument to `.get()` (just `customer.get("email")`, no de-

fault) — this doesn't error; it silently returns `None` instead. Demonstrate this live: `print(customer.get("email"))` alone prints `None`, which *looks* like a reasonable, quiet result but may not actually be what a program downstream expects — a good moment to reinforce that `None` is a real, distinct value, not the same as “nothing happened.”

- Using `.get()` everywhere out of an instinct to “always be safe,” even in situations where a missing key genuinely should be treated as an error worth surfacing loudly — worth a brief counter-example: if `customer["name"]` were somehow missing, silently defaulting to, say, `"Unknown"` might hide a real data problem that a `KeyError` would have caught immediately during testing.

Check for understanding: “In Exercise 6, you’ll loop through several different customer dictionaries that might not all have identical fields. Which lookup style — `.get()` with a default, or bracket notation — would you reach for by default in that situation, and why?” (`.get()` with a sensible default — since looping over multiple records where structure might vary slightly is exactly the “missing key is a normal, expected possibility” case the decision rule above describes; getting a student to apply the rule to a near-future exercise, rather than just restate it, confirms it actually transferred.)

6.5 Exercise 4 — Iterate (0:27–0:34, 7 min)

Teaching goal: Loop over a dictionary's contents with `.items()`, and understand what iterating *without* `.items()` gives you instead — keys only.

Say to the class:

“Looping over a dictionary needs one new piece of syntax: `.items()`, to get both the key and the value together on each pass.”

Live-code this:

```
# --- Exercise 4 ---
for key, value in customer.items():
    print(f"{key}: {value}")

print("--- without .items() ---")
for key in customer:
    print(key)
```

Line-by-line explanation:

- `for key, value in customer.items():` — `.items()` gives back each key/value pair together, and the `for key, value in ...` syntax **unpacks** each pair into two loop variables in one step — the exact same unpacking idea as Module 07 Exercise 4's `for price, tier in orders:` over a list of tuples (worth naming that connection explicitly: a dictionary's `.items()` behaves like a list of key/value tuples for looping purposes).
- `f"{key}: {value}"` — produces the exact "name: Alice"-style formatted line the exercise asks for, reusing ordinary f-string substitution.
- `for key in customer:` (no `.items()`) — looping over a dictionary **directly**, with no `.items()` call, gives you only the **keys**, one per pass — not the values, and not pairs. This is worth demonstrating explicitly side by side with the `.items()` version, since it's genuinely easy to assume a bare `for x in some_dict:` would give you the values, or pairs, when it actually only ever gives keys.

Run it. Expected output:

```
name: Alice
tier: platinum
ytd_spend: 1200
phone: 555-1234
--- without .items() ---
name
tier
ytd_spend
```

phone

Have students write the required explanatory comment — the exercise’s actual deliverable — describing the difference: iterating without `.items()` gives keys only; with `.items()` gives both keys and values together.

Common student mistakes to watch for:

- Writing `for key, value in customer:` (forgetting `.items()` but still trying to unpack two variables) — raises `ValueError: too many values to unpack (expected 2)`, since a bare dictionary iteration yields only single keys, not pairs, and Python can’t unpack one key-string into two variables. A good, very literal error message worth reading together.
- Assuming the printed key order is alphabetical or otherwise automatically “organized” — it’s actually **insertion order**: `name` and `tier` print first here specifically because they were the first fields ever added (back in Exercise 1), and `phone` prints last because it was the most recently added field (Exercise 2) — the order tracks *when each key was first inserted*, not the alphabet, not the order keys were last modified, and not any other implicit sorting. If a student expects alphabetical order, clarify explicitly that dictionaries don’t sort automatically — `sorted()` is a separate, deliberate step, as Exercise 8 will show.

Check for understanding: “If I wanted keys sorted alphabetically instead of in insertion order, what one built-in function could help?” (`sorted()` — e.g. `for key in sorted(customer):` — a good callback confirming `sorted()` works generally on any iterable, not just lists, which previews Exercise 8’s own use of `sorted()` on dictionary keys later in this same lab.)

6.6 Exercise 5 — Lookup Table (0:34–0:42, 8 min)

Teaching goal: Use a dictionary as a **lookup table** — mapping a small, fixed set of categories (tier names) to associated values (discount rates) — a genuinely different *use* of a dictionary than Exercises 1–4’s single-customer record.

Say to the class:

“New use for a dictionary: not a record about one thing, but a lookup table mapping a category name to a value — tier name to discount rate. This is one of the most common, useful dictionary patterns in real business code.”

Live-code this:

```
# --- Exercise 5 ---
tier_discounts = {"bronze": 0, "silver": .05, "gold": .10, "platinum": .15}

discount = tier_discounts.get(customer["tier"], 0)
final_price = 200 * (1 - discount)
print(f'Tier: {customer["tier"]} | Discount: {discount*100:.0f}% | Final: ${final_price:.2f}')
```

Line-by-line explanation:

- `tier_discounts = {"bronze": 0, "silver": .05, ...}` — a dictionary that isn’t describing one specific thing (like `customer` describes Alice) — it’s a **general-purpose mapping**, usable for looking up *any* customer’s discount rate, given their tier name.
- `tier_discounts.get(customer["tier"], 0)` — this line does two lookups in sequence, worth reading inside-out: first, `customer["tier"]` retrieves *this specific customer’s* tier string ("platinum", after Exercise 2’s update); then, `tier_discounts.get(...)` uses that string as the *key* to look up the corresponding discount rate in the lookup table. The default of `0` here is a deliberate safety net: if a customer’s tier were somehow something not in the lookup table at all (a typo, a new tier not yet added to `tier_discounts`), this returns `0` — no discount — rather than crashing the whole program. Connect this explicitly back to Exercise 3’s `.get()`-vs-bracket-notation decision rule: an unrecognized tier is exactly the kind of “expected possibility, handle it gracefully” situation `.get()` is well suited for.
- `f"...{customer['tier']}..."` — note the **single quotes** around `tier` inside an f-string that’s itself wrapped in **double quotes** — Python requires the inner and outer quote characters to differ (or requires escaping) to avoid ambiguity about where the string actually ends; worth a ten-second note if a student’s editor flags this or asks about it.

Run it. Expected output (continuing with Alice, now platinum tier from Exercise 2):

```
Tier: platinum | Discount: 15% | Final: $170.00
```

Common student mistakes to watch for:

- Using bracket notation (`tier_discounts[customer["tier"]]`) instead of `.get()` — works fine as long as every possible tier is guaranteed present in `tier_discounts`, but loses the safety net for any typo or future new tier; worth asking the room to articulate why `.get()` is the more defensive choice here specifically, even though bracket notation would produce identical output for this exact data.
- Confusing `tier_discounts` (the lookup table, shared across all customers) with `customer` (one specific record) — a genuinely common naming/purpose mix-up early in this exercise; a quick “which of these two dictionaries describes *one person*, and which describes *a rule that applies to everyone*?” question resolves it quickly.

Check for understanding: “If a new tier called 'diamond' were introduced with a 20% discount, what's the *only* line that needs to change in this script?” (Just the `tier_discounts` dictionary literal — add `"diamond": .20` — nothing about the lookup logic itself needs to change, since `.get()` already handles whatever key it's given. This is the real payoff of a lookup table: business rules can change without rewriting logic.)

6.7 Exercise 6 — List of Dicts (0:42–0:50, 8 min)

Teaching goal: Combine Module 10's lists with this module's dictionaries — a **list of dictionaries**, the single most common shape for representing a table of records in Python, and loop over it applying Exercise 5's lookup-table pattern to each one.

Say to the class:

"This is the shape you'll see constantly in real data work: a list, where every item is itself a dictionary with the same fields — like rows in a spreadsheet, where each row is a customer."

Live-code this:

```
# --- Exercise 6 ---
customers = [
    {"name": "Alice", "tier": "gold"},
    {"name": "Bob", "tier": "silver"},
    {"name": "Carol", "tier": "bronze"},
]

for c in customers:
    discount = tier_discounts.get(c["tier"], 0)
    print(f"{c['name']} | {c['tier']} | discount: ${300 * discount:.2f}")
```

Line-by-line explanation:

- `customers = [{...}, {...}, {...}]` — a list (square brackets) whose elements are dictionaries (curly braces) — say explicitly: this is nesting one data structure inside another, and it's worth pointing at the punctuation directly on screen: the outer `[ ]` make a list; each inner `{ }` makes one dictionary, and they're separated by commas exactly like any other list of values.
- `for c in customers:` — a completely ordinary loop over a list, from Module 06 — the only thing new is that each `c` is a dictionary, not a plain number or string, so accessing its fields inside the loop needs dictionary syntax (`c["name"]`, `c["tier"]`), not anything new about the loop itself.
- `tier_discounts.get(c["tier"], 0)` — Exercise 5's lookup-table pattern, now applied inside a loop, once per customer — say explicitly that this is the exact same one-line pattern reused three times automatically by the loop, rather than being retyped for Alice, then Bob, then Carol individually.

Run it. Expected output:

```
Alice | gold | discount: $30.00
Bob | silver | discount: $15.00
Carol | bronze | discount: $0.00
```

Common student mistakes to watch for:

- Forgetting a comma between dictionary entries in the customers list literal — a `SyntaxError`, same category as a missing comma in any list; worth a quick visual check of the punctuation if it comes up.
- Reaching for index-based access (`customers[0]["name"]`) inside the loop instead of the loop variable `c["name"]` — not wrong exactly if written outside a loop, but defeats the purpose of looping at all if used *inside* one; a good moment to ask “why would hardcoding an index inside this loop not make sense?”

Check for understanding: “If Carol’s tier were misspelled as 'Bronze' (capital B) in her dictionary, what would this script print for her discount, and would it error?” (`$0.00`, silently — `.get()`’s case-sensitive lookup wouldn’t match “bronze” in the lookup table, falls through to the default `0`, and produces a plausible-looking but wrong result with no error at all. A good “silent wrong answer” example, directly connecting Exercise 5’s safety-net framing to a genuine real risk of that same safety net: `.get()`’s graceful fallback can also gracefully hide a real data-entry mistake.)

6.8 Exercise 7 — Nested Catalog (0:50–0:58, 8 min)

Teaching goal: A dictionary whose *values* are themselves dictionaries — genuinely nested data, one level deeper than Exercise 6's list-of-dicts — and looping through it to apply a filtering condition.

Say to the class:

“One more level of nesting: a dictionary where each value is itself a whole dictionary of details. A product catalog, keyed by product name, where each product has its own price and stock count.”

Live-code this:

```
# --- Exercise 7 ---
catalog = {
    "Widget A": {"price": 9.99, "stock": 150},
    "Gadget C": {"price": 49.99, "stock": 8},
}

for product, info in catalog.items():
    if info["stock"] < 20:
        print(f"LOW STOCK: {product} ({info['stock']} left)")
```

Line-by-line explanation:

- `catalog = {"Widget A": {...}, "Gadget C": {...}}` — the **outer** dictionary's keys are product names (strings); the outer dictionary's **values** are themselves dictionaries, each with their own price/stock keys. Point at the nesting explicitly on screen: two closing `}}` at the end of each product's entry — one closes the inner price/stock dictionary, the other (eventually, after the whole catalog) closes the outer one.
- `for product, info in catalog.items():` — Exercise 4's `.items()` pattern again: `product` is bound to each outer key (a product name string), `info` is bound to each outer value — which, this time, is itself a whole dictionary, not a plain number or string.
- `if info["stock"] < 20:` — reaching *into* the inner dictionary (`info`), using the exact same bracket notation as any other dictionary access — say explicitly: there's nothing structurally new here; `info` is just a dictionary like any other, it happens to be a value living inside a larger one.
- `info['stock']` inside the f-string — same single-quote-inside-double-quote pattern flagged in Exercise 5.

Run it. Expected output (only one product triggers the alert):

LOW STOCK: Gadget C (8 left)

Common student mistakes to watch for:

- Trying `catalog["price"]` directly (skipping the product name) — raises `KeyError`, since "price" isn't a key of the *outer* dictionary at all; it's a key of one of the *inner* dictionaries. Getting a student to trace exactly which dictionary they're currently "inside" of at each bracket step is the real skill this mistake reveals a gap in.
- Confusing `product` and `info` — e.g. printing `product['stock']` instead of `info['stock']` — since `product` is just a string (a product name), attempting to index into it with a string key like 'stock' raises a `TypeError`, worth reading together if it comes up, since it's a good illustration that a string doesn't support key-based lookup the way a dictionary does.

Check for understanding: "How would you print every product's stock level, not just the low ones — what's the minimal change?" (Remove the `if info["stock"] < 20:` line entirely, or replace the print statement's content — either way, get a student to articulate that the `if` is purely a *filter* layered on top of an otherwise ordinary full iteration, the same "loop first, filter second" structure as Module 10 Exercise 8.)

6.9 Exercise 8 — Sales by Region (0:58–1:07, 9 min)

Teaching goal: The most sophisticated pattern in this lab — build a **summary dictionary** from a list of transaction records, accumulating totals *by key*, using `.get(key, 0)` as the accumulator’s safe starting point. This is a direct, real preview of what a `GROUP BY` does in SQL or a pivot table does in Excel.

Say to the class:

“Last pattern of the day, and it’s the one you’ll use constantly in real analytics work: given a list of transactions, each tagged with a region, build a summary — one total per region — with a loop. This is exactly what a pivot table or a SQL `GROUP BY` does under the hood.”

Live-code this:

```
# --- Exercise 8 ---
transactions = [
    {"region": "South", "amount": 300},
    {"region": "North", "amount": 150},
    {"region": "South", "amount": 200},
]

summary = {}                                # initialize – an empty dict this time
for t in transactions:
    region = t["region"]
    summary[region] = summary.get(region, 0) + t["amount"]    # update

for region in sorted(summary):
    print(f"{region}: {summary[region]}")
```

Line-by-line explanation:

- `summary = {}` — **initialize**, same accumulator-pattern skeleton from Module 06, but the “empty” starting value is now an empty *dictionary*, not `0` or `[]` — a third shape for the same underlying pattern this course keeps returning to.
- `summary[region] = summary.get(region, 0) + t["amount"]` — **this is the single most important line in this lab; walk it slowly, right to left.** `summary.get(region, 0)` looks up the running total so *far* for this region — using `.get()` with a default of `0` specifically because, on a region’s *first* appearance, it isn’t in `summary` yet at all, and without the default, this would raise `KeyError` immediately. `+ t["amount"]` adds this transaction’s amount onto whatever that running total was. `summary[region] = ...` (plain bracket-notation assignment, not `.get()`) stores the new running total back — say explicitly why the *assignment* side uses bracket notation while the *read* side uses `.get()`: assignment always works regardless of whether the key existed before (Exercise 2’s “same syntax adds or updates” lesson), so

there's no need for a default there — only the *read*, which happens before we know if this region has been seen yet, needs the safety net.

- **Trace it by hand with the class, transaction by transaction, since this is the real payoff:** first transaction, South/300 — `summary.get("South", 0)` is 0 (not seen yet), so `summary["South"] = 0 + 300 = 300`. Second, North/150 — `summary.get("North", 0)` is 0, so `summary["North"] = 150`. Third, South/200 again — `summary.get("South", 0)` is now 300 (found this time!), so `summary["South"] = 300 + 200 = 500`.
- `for region in sorted(summary):` — `sorted()` on a dictionary sorts and returns its **keys** (region names), alphabetically — reusing Exercise 4's idea that `sorted()` works on more than just lists.

Run it. Expected output:

North: 150

South: 500

Common student mistakes to watch for:

- Writing `summary[region] += t["amount"]` directly, without the `.get()` safety net at all — this fails on a region's *first* appearance specifically, with `KeyError`, since `+=` requires the key to already exist (it's shorthand for `summary[region] = summary[region] + ...`, and the right-hand `summary[region]` read has no default to fall back on). This is worth demonstrating live as the natural-but-broken first instinct, since `+=` has worked everywhere else all semester up to this point — the dictionary accumulator case is genuinely the first place it silently doesn't just work without the extra `.get()` step.
- Forgetting the default entirely (`summary.get(region)`, no 0) — returns `None` on a region's first appearance, and `None + t["amount"]` raises `TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'` — a good, readable error worth tracing back to its cause together.

Check for understanding: “If a fourth transaction came in for a brand-new region, 'East', what's the very first value `summary.get('East', 0)` would return, and why does that matter?” (0 — the default, since 'East' has never been seen before this point; getting a student to state explicitly *why* 0 is the correct starting value for a running sum (not 1, not the transaction amount itself) confirms the accumulator logic, not just the syntax, actually landed.)

7 Wrap-Up (last ~8 minutes)

Review the reflection questions out loud (answered as a comment at the top of the file, per this lab's submission format):

1. *When to use `.get()` vs. bracket notation, and your rule of thumb* — a strong answer restates the decision rule from Exercise 3 in the student's own words, with a concrete example of each: `.get()` for an expected-possibly-missing field you want to handle gracefully; bracket notation when a missing key should be a loud, immediate signal something's wrong.
2. *What Exercise 8's summary-dictionary pattern reminds you of from Excel* — push for the specific term if a student knows it: a **pivot table**, or a SUMIF/GROUPBY-style aggregation — and a genuine reason the Python version scales better: no manual dragging of fields, works identically whether the list has 3 transactions or 3 million.

Review the submission checklist together:

- ☐ File is named `customers.py`
- ☐ Contains Exercises 1–8, each clearly separated
- ☐ Reflection comment at the top of the file, answering both questions
- ☐ Pushed to GitHub inside a `week11/` folder
- ☐ Repo URL submitted to Canvas

Preview Module 12: “Every dictionary and list today lived only inside your running script — the moment the program ends, it's gone. Next module, you learn to read and write actual files, so the data your programs work with can outlive the program itself.”

8 Appendix A — Full Answer Key (`customers.py`)

```
# customers.py
# ISM2411 Module 11 Lab – Customer Dictionary & Lookup
# Reflection:
# 1. [.get() vs bracket notation – rule of thumb, student's own words]
# 2. [Excel comparison for Exercise 8's summary pattern]

# --- Exercise 1 ---
customer = {
    "name": "Alice",
    "email": "alice@example.com",
    "tier": "gold",
    "ytd_spend": 1200,
}
print(customer["name"])
```

```

print(customer["email"])
print(customer["tier"])
print(customer["ytd_spend"])
print(customer)

# --- Exercise 2 ---
customer["phone"] = "555-1234"
print(customer)
customer["tier"] = "platinum"
print(customer)
del customer["email"]
print(customer)

# --- Exercise 3 ---
print(customer.get("email", "no email on file"))
try:
    print(customer["email"])
except KeyError as e:
    print(f"KeyError caught: {e}")

# --- Exercise 4 ---
for key, value in customer.items():
    print(f"{key}: {value}")
# Without .items(), iterating a dict gives keys only, not values.
for key in customer:
    print(key)

# --- Exercise 5 ---
tier_discounts = {"bronze": 0, "silver": .05, "gold": .10, "platinum": .15}
discount = tier_discounts.get(customer["tier"], 0)
final_price = 200 * (1 - discount)
print(f"Tier: {customer['tier']} | Discount: {discount*100:.0f}% | Final: ${final_price:.2f}")

# --- Exercise 6 ---
customers = [
    {"name": "Alice", "tier": "gold"},
    {"name": "Bob", "tier": "silver"},
    {"name": "Carol", "tier": "bronze"},
]
for c in customers:
    discount = tier_discounts.get(c["tier"], 0)

```

```

    print(f"{c['name']} | {c['tier']} | discount: ${300 * discount:.2f}")

# --- Exercise 7 ---
catalog = {
    "Widget A": {"price": 9.99, "stock": 150},
    "Gadget C": {"price": 49.99, "stock": 8},
}
for product, info in catalog.items():
    if info["stock"] < 20:
        print(f"LOW STOCK: {product} ({info['stock']} left)")

# --- Exercise 8 ---
transactions = [
    {"region": "South", "amount": 300},
    {"region": "North", "amount": 150},
    {"region": "South", "amount": 200},
]
summary = {}
for t in transactions:
    region = t["region"]
    summary[region] = summary.get(region, 0) + t["amount"]
for region in sorted(summary):
    print(f"{region}: {summary[region]}")

```

9 Appendix B — Extra Practice (only if the class finishes early)

Eight required exercises fill the full 75 minutes at a normal pace. If a section moves unusually fast:

Extra — a second lookup table, different domain. `shipping_speeds = {"standard": 5, "express": 2, "overnight": 1}` (values are days). Have students write a one-line lookup, using `.get()` with a default of 7, for a speed variable set to "express", then again for "same-day" (not in the table) to confirm the default fires correctly. (2, then 7.)

Extra — a second accumulate-into-a-dict exercise. `orders = [{"category": "Books", "amount": 45}, {"category": "Electronics", "amount": 320}, {"category": "Books", "amount": 15}, {"category": "Toys", "amount": 60}]`. Have students build a `category_totals` summary dictionary using Exercise 8's exact pattern, then print it sorted by category name. (Books: 60, Electronics: 320, Toys: 60 — note Books and Toys land on the same total, a good moment to point out that's a coincidence of this specific data, not a bug, if a student flags it as suspicious.)